

# **Operating Systems Design**

**Course Code: CS207**

**Unit 2**

**Dr. Gunjan Chugh**

**Assistant Professor**

**Dept. of Computer Science Engineering  
Delhi Technological University**

# Contents of Unit 2

- ❑ Process concept,
- ❑ Principle of Concurrency,
- ❑ Producer Consumer Problem,
- ❑ Critical Section problem,
- ❑ Semaphores, Classical problems in Concurrency,
- ❑ Inter Process Communication,
- ❑ Process Generation, Process Scheduling.
- ❑ CPU Scheduling: Scheduling Concept, Performance Criteria of Scheduling Algorithm,
- ❑ Multiprocessor Scheduling.

# What is a Process?

- ❑ **Program:** It is a set of instructions.
- ❑ **Process:** A program in execution.
- ❑ Program becomes process when executable file loaded into memory
- ❑ **Program** is **passive entity** stored on disk (executable file), **the process is active**
- ❑ Execution of program started via GUI mouse clicks, command line entry of its name, etc
- ❑ One program can be several processes
  - ❑ For e.g. multiple users executing the same program

# CPU-Bound vs I/O-Bound Processes

- ❑ A CPU-bound process requires more CPU time or spends more time in the running state.
- ❑ An I/O-bound process requires more I/O time and less CPU time.
- ❑ An I/O-bound process spends more time in the waiting state.
- ❑ The goal of process scheduling is to improve overall system performance by maximizing CPU utilization, minimizing throughput time, and improving system response time.

# Process Management

## 1. Process Creation and Termination

- Process creation involves creating a Process ID, setting up Process Control Block, etc. A process can be terminated either by the operating system or by the parent process. Process termination involves clearing all resources allocated to it.

## 2. CPU Scheduling

- In a multiprogramming system, multiple processes need to get the CPU. It is the job of Operating System to ensure smooth and efficient execution of multiple processes.

# Process Management

## 3. Deadlock Handling

- Making sure that the system does not reach a state where two or more processes cannot proceed due to cyclic dependency on each other.

## 4. Inter-Process Communication

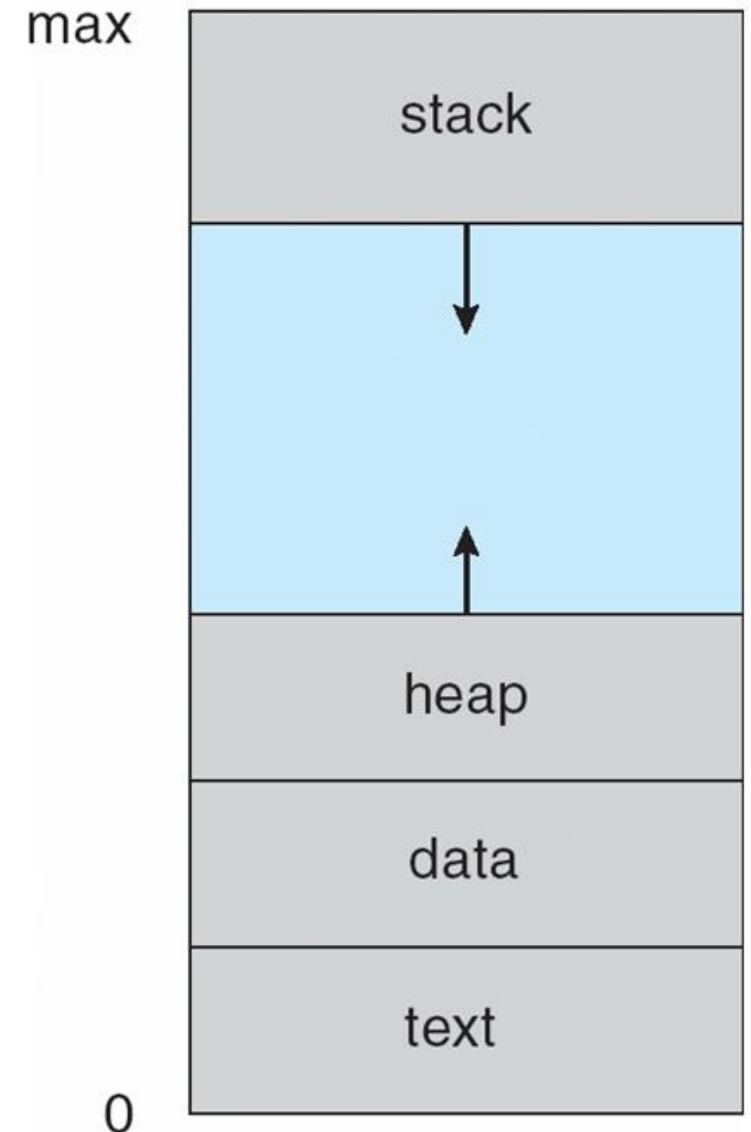
Operating System provides facilities such as shared memory and message passing for cooperating processes to communicate.

## 5. Process Synchronization

Process Synchronization is the coordination of execution of multiple processes in a multiprogramming system to ensure that they access shared resources (like memory) in a controlled and predictable manner.

# Process in Main Memory

- ❑ **Text Section:** Contains the program code
- ❑ **Stack:** contains temporary data like Function parameters, return addresses, local variables.
- ❑ **Data section:** contains global variables
- ❑ **Heap:** contains memory dynamically allocated during run time



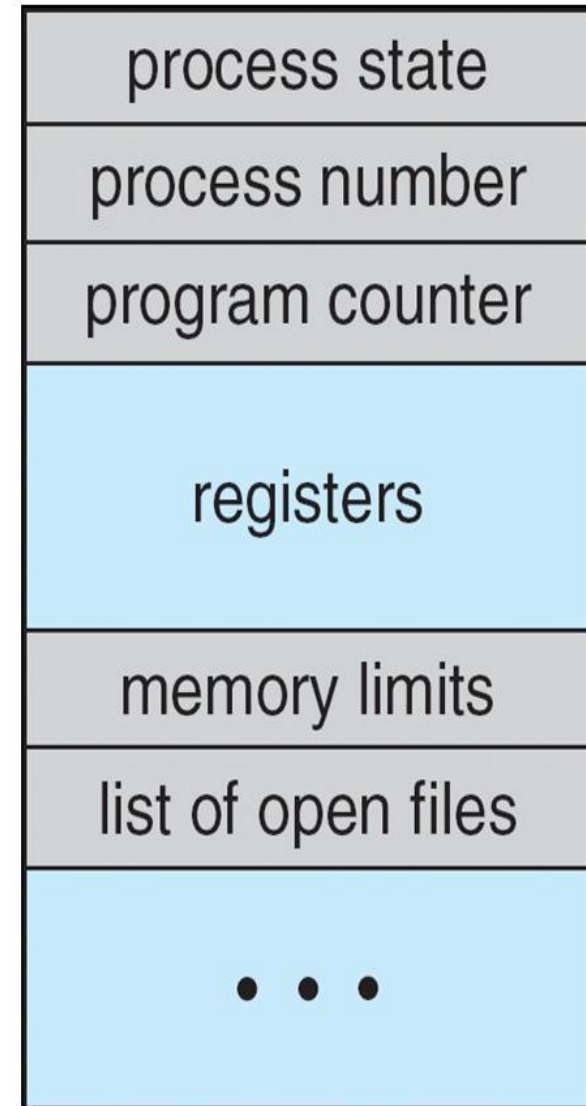
# Attributes of a Process

- A **process** has several important attributes that help the operating system manage and control it.
- These attributes are stored in a structure called the **Process Control Block (PCB)** (sometimes called a task control block).
- The PCB keeps all the key information about the process

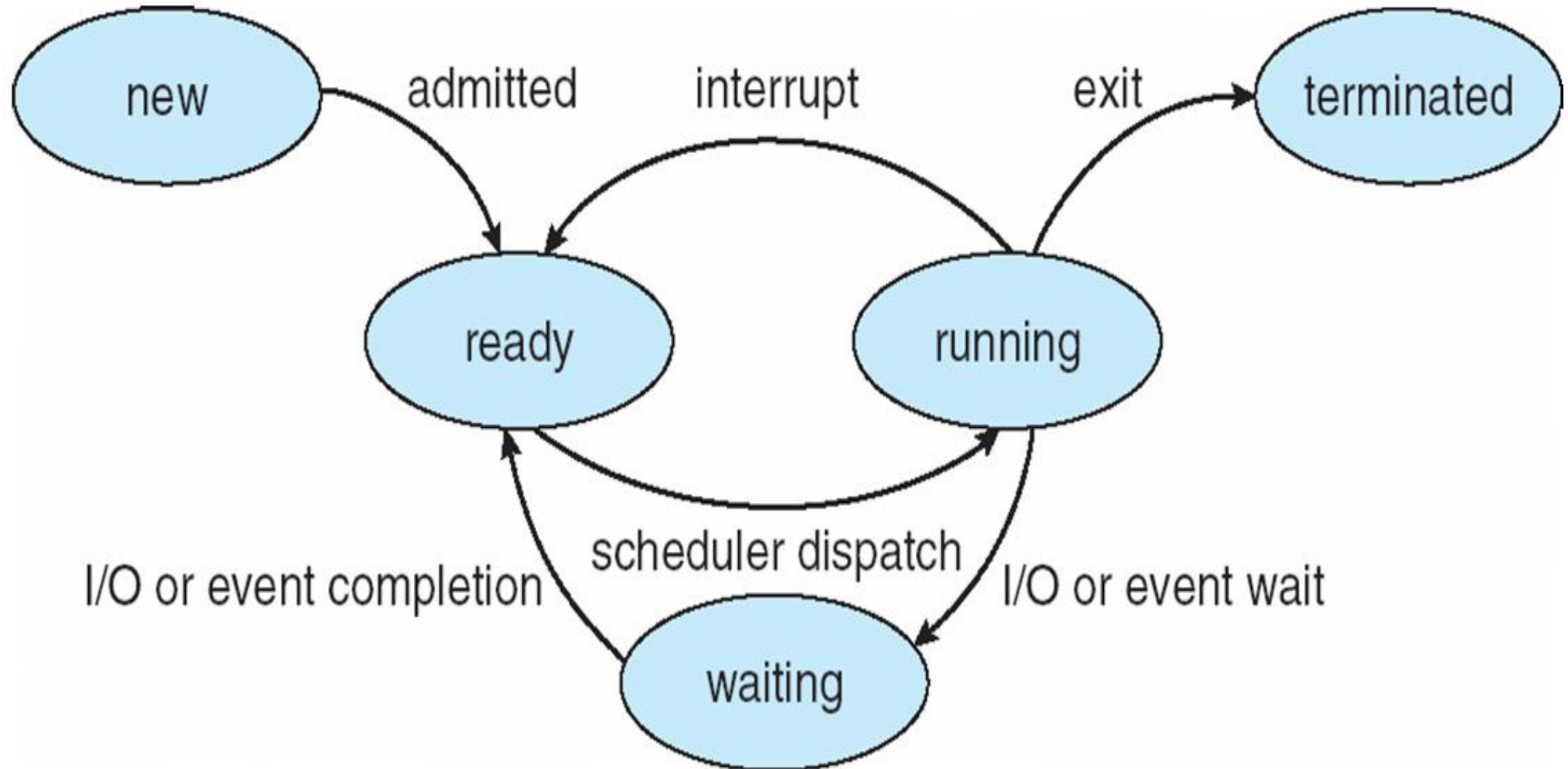
# Process Control Block (PCB)

Information associated with each process  
(also called **task control block**)

- ❑ **Process ID (PID):** A unique number assigned to each process
- ❑ **Process state** – running, waiting, etc
- ❑ **Program counter** – location of instruction to next execute
- ❑ **CPU registers** – contents of all process-centric registers
- ❑ **CPU scheduling information-** priorities, scheduling queue pointers
- ❑ **Memory-management information** – memory allocated to the process
- ❑ **Accounting information** – CPU used, clock time elapsed since start, time limits
- ❑ **I/O status information** – I/O devices allocated to process, list of open files



# Life Cycle of a Process



# Process States

**As a process executes, it changes from one state to another.**

- ❑ **new:** The process is being created
- ❑ **running:** Instructions are being executed
- ❑ **waiting:** The process is waiting for some event to occur
- ❑ **ready:** The process is waiting to be assigned to a processor
- ❑ **terminated:** The process has finished execution

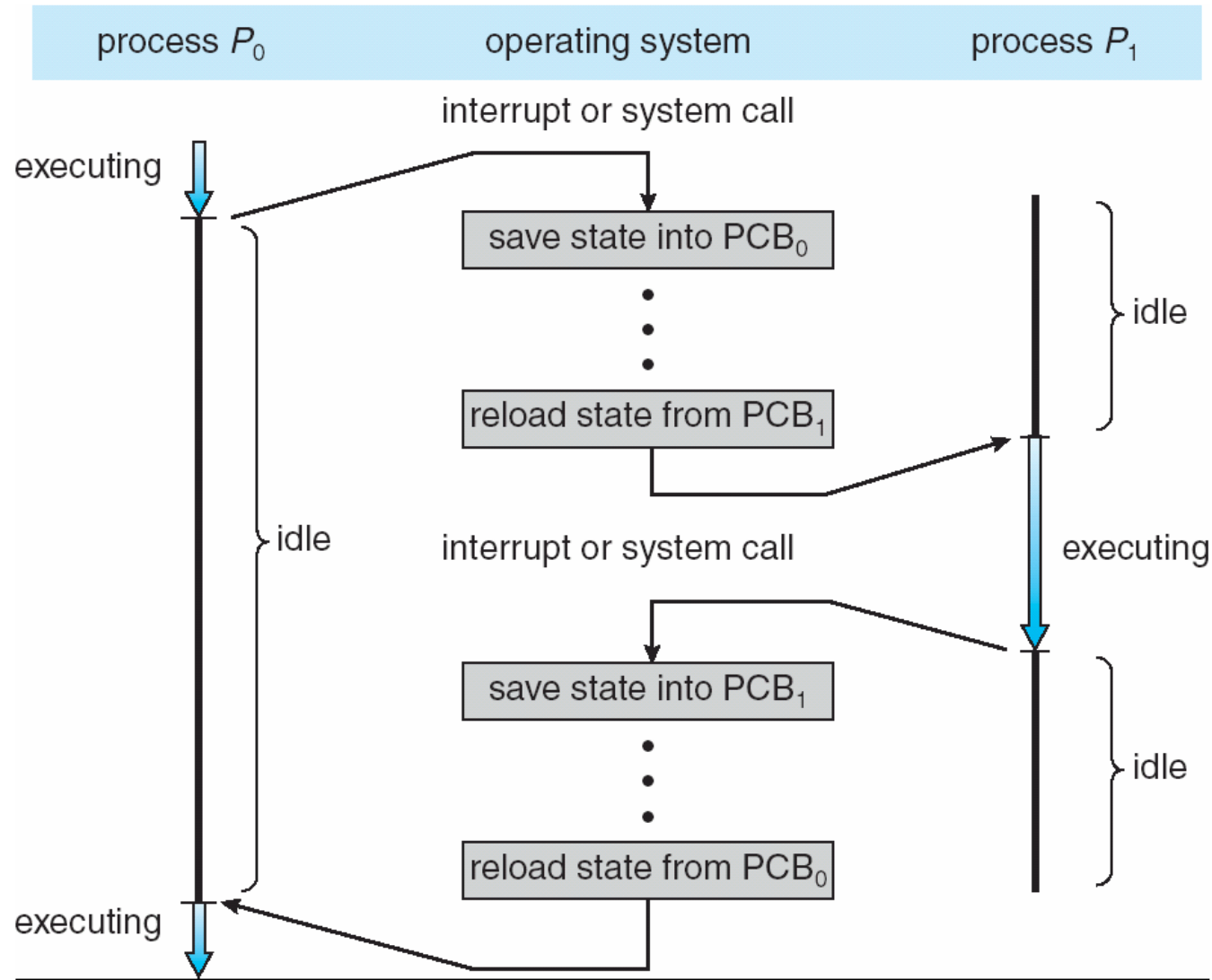
# Context Switching

- ☐ **When CPU switches to another process, the system must save the state of the old process and load the saved state for the new process via a context switch.**
- ☐ Context of a process represented in the PCB
- ☐ Context-switch time is overhead; the system does no useful work while switching
- ☐ The dispatcher is the component that actually performs the switch.

# Context Switching Triggers

- ❑ **Interrupts:** When a CPU requests that data be read from a disc, if any interruptions occur, context switching automatically switches to a component of the hardware that can handle the interruptions more quickly.
- ❑ **Multitasking:** When a process is switched, the previous state is retained so that the process can continue running at the same spot in the system.
- ❑ **Kernel/User Switch:** This trigger is used when the OS needed to switch between the user mode and kernel mode.

# CPU switching from Process to Process



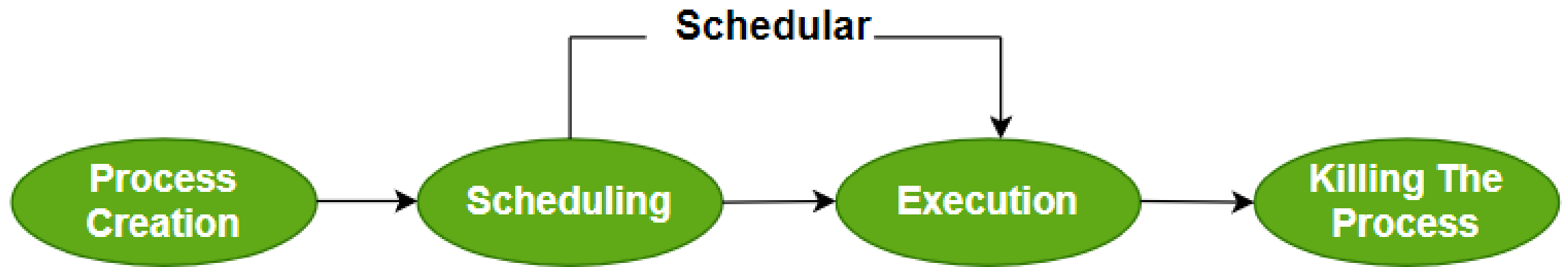
# When Does Context Switching Happen?

- When a high-priority process comes to a ready state (i.e., with a higher priority than the running process).
- An Interrupt occurs.
- User and kernel-mode switch
- Preemptive CPU scheduling is used.

# Factors On Which Context Switch Time Depends

- **Hardware support** (registers, instructions).
- **Size of process context** (registers, memory tables).
- **Mode switching cost.**
- **Cache/TLB invalidation.**
- **OS kernel implementation.**

# Process Operations



# PROCESS CREATION

## 1. Parent Process Calls To Create Process

- A parent process calls a system call to create a new process.

**In Unix/Linux:** `fork()`

**In Windows:** `CreateProcess()`

- This is typically done when a user starts a program or a process spawns a child.

## 2. Allocate Memory and Resources

- The OS allocates memory for the new process in both the code and data segments.
- It also allocates other required resources such as: File descriptors, I/O buffers & Stack and heap space.

### **3. Create a Unique Process Control Block (PCB)**

- The OS creates a Process Control Block for the new process.

### **4. Initialize the Process Context**

- The new process inherits or initializes: Register values, Program counter (PC) set to start of program, Stack pointer initialized & File table copied (if needed).
- In some systems (fork()), the child gets a copy of the parent's context.

## 5. Link Process in OS Structures

- The OS links the new process into:
  - Job queue
  - Ready queue
  - Parent-child hierarchy

## 6. Set Process State to READY

- The new process is marked as READY and placed in the ready queue.
- It will wait for the CPU scheduler to dispatch it.

## 7. Scheduler Picks the Process to Run

- The CPU scheduler eventually selects the new process from the ready queue.
- It dispatches the process to the CPU, changing its state to RUNNING.

# PROCESS DELETION

- **Processes terminate themselves when they finish executing their last statement, after which the operating system uses the `exit()` system call to delete their context.**
- Then all the resources held by that process like physical and virtual memory, buffers, open files, etc., are taken back by the operating system.
- A process P can be terminated either by the operating system or by the parent process of P.

# PROCESS DELETION

- A parent may terminate a process due to one of the following reasons:
  - When task given to the child is not required now.
  - When the child has taken more resources than its limit.
  - The parent of the process is exiting, as a result, all its children are deleted. This is called cascaded termination.

# PREEMPTIVE & NON-PREEMPTIVE SCHEDULING

## 1. Preemptive Scheduling

- In preemptive scheduling, the operating system can **interrupt** a currently running process in order to assign the CPU to another process.

## 2. Non-Preemptive Scheduling

- In non-preemptive scheduling, once a process starts its execution on the CPU, it runs to **completion** or until it voluntarily releases the CPU (for example, by waiting for an I/O operation).
- The operating system **cannot** interrupt a running process.

# KEY DIFFERENCES

| Feature          | Preemptive Scheduling                                   | Non-Preemptive Scheduling                           |
|------------------|---------------------------------------------------------|-----------------------------------------------------|
| Interruptions    | Can interrupt a running process at any time             | The process runs to completion once started         |
| Complexity       | More complex due to context switching and interruptions | Simpler, as no interruptions occur during execution |
| CPU Allocation   | The OS can take back the CPU from a process             | Once a process starts, it cannot be preempted       |
| System Overhead  | Higher, due to context switching                        | Lower, as there are fewer context switches          |
| Suitable Systems | Time-sharing, multi-user, or interactive systems        | Batch processing, or simple systems                 |
| Responsiveness   | High, as processes are given equal CPU time             | Can be low, especially if a long process dominates  |